Privacy Preserving Smart Water

Fully Homomorphic Encryption for Smart Water Meter Analytics
Senior Design Project Report — Revised Edition

Ben Antonellis · Gabe Smith · Alex Nguyen

Sponsor: Dr. Shameek Bhattacharjee, Western Michigan University
Faculty advisors: Halil Dursunoglu, Dr. Wuwei Shen

Original report: Fall 2024 · Revised: October 2026
Code: github.com/alexworkspace1/fully-homomorphic-smart-water-meter
(originally github.com/Linnydude3347/SmartWaterMeter)

Contents

Preface

1. Introduction

2. Glossary

3. User Requirements

4. System Requirements

5. System Architecture

   5.1 System context · 5.2 Components · 5.3 Keys and trust boundaries · 5.4 Protocol · 5.5 Lookup tables and PIR · 5.6 Deployment of the simulation

6. System Models

   6.1 Notation · 6.2 Statistical model · 6.3 Lookup-table model · 6.4 Polynomial representation · 6.5 Encoding and precision · 6.6 Error budget · 6.7 FHE parameters and noise · 6.8 Threat model

7. System Evolution

8. Testing and Validation

9. Conclusion

Appendices A–E

Bibliography

> **About this edition.** This revision keeps the original report's structure and findings. It adds a full system architecture with diagrams, formal system models, the project's evolution and design decisions, a testing chapter and a conclusion with limitations and future work. Where the original report and the code differ, this edition describes the code and notes the difference (Appendix D).

Preface

The way we monitor and manage vital resources has changed dramatically as cutting-edge technologies have been built into critical infrastructure such as water distribution networks. "Smart water grids", like their smart energy grid counterparts, are a way to make water supply systems more sustainable, dependable and efficient. They use data analytics and Internet of Things (IoT) devices to optimise distribution, find leaks, resist cyber attacks and improve system performance.

This report examines one technology for such grids: fully homomorphic encryption (FHE). FHE protects the sensitive data that moves through smart water infrastructure while still allowing analytics to run on that data in encrypted form. The introduction describes the project's goals, motivation and contributions.

1. Introduction

Communities depend on water delivery systems to supply clean water for drinking, sanitation and industry. As in electrical grids, Advanced Metering Infrastructure (AMI) is essential to monitoring and controlling consumption in modern smart water systems. These grids combine smart meters at each customer's location with communication networks and data management systems.

Like electrical grids, smart water networks face **data integrity threats** that undermine the accuracy of usage data:

- **Deductive attacks** report less consumption than was used, the water-grid form of theft.
- **Additive attacks** report more consumption than was used, raising customers' bills and misdirecting resources.
- **Camouflage attacks** combine the two, raising some meters and lowering others so that mean consumption stays the same while individual records change.

Detecting these attacks requires analytics over many customers' readings. Conventional systems need access to each customer's consumption, which raises privacy concerns, may conflict with privacy law and can damage customer trust. Secure Multiparty Computation and Differential Privacy offer theoretical privacy guarantees but often struggle with scalability or accuracy. Fully Homomorphic Encryption computes directly on encrypted data, preserving both privacy and the exactness of the result.

This project builds a privacy-preserving analytics system for smart water grids, adapted from earlier work on smart energy grids. It combines the BFV homomorphic encryption scheme with a **Look-Up Table (LUT)** approach and **Private Information Retrieval (PIR)** to compute, on encrypted data, the daily ratio of the Harmonic Mean to the Arithmetic Mean of a region's consumption. This ratio is a known indicator of false data injection. The chapters that follow describe the requirements, the architecture, the models behind each computation, how the system evolved, how it was tested, and what it achieved.

2. Glossary

Term	Meaning
Arithmetic Mean (AM)	The sum of values divided by their count.
Advanced Metering Infrastructure (AMI)	Smart meters, communication networks and data management systems that let a utility monitor and manage consumption remotely.
Batching / SIMD	Packing many values into the slots of one ciphertext so that one operation acts on all of them (Single Instruction, Multiple Data).
Brakerski-Fan-Vercauteren (BFV)	An FHE scheme for exact addition, subtraction and multiplication of integers modulo a plain modulus.
Computation Server (CS)	The party that performs every homomorphic computation. It never holds the secret key.
Differential Privacy (DP)	Protecting individuals in a data set by adding calibrated noise to the results of an analysis.
Fully Homomorphic Encryption (FHE)	Encryption that allows arbitrary computation on ciphertexts without decrypting them.
Galois keys	Evaluation keys that let the CS rotate the slots of a ciphertext.
Harmonic Mean (HM)	The count of values divided by the sum of their reciprocals. For positive values, HM ≤ AM.
Homomorphic Encryption (HE)	Encryption that supports some algebraic operations on ciphertexts.
Internet of Things (IoT)	Networked devices with sensors and software that collect and exchange data.
Look-Up Table (LUT)	A pair of encrypted tables, input values and their outputs, used to evaluate a function BFV cannot compute directly.
Multiplicative depth	The longest chain of ciphertext multiplications in a computation. It sets the noise budget needed.
Noise budget	How much more noise a ciphertext can take before it no longer decrypts correctly.
Plain modulus	The modulus of the integers BFV encrypts (786433 here). Every encoded value must stay below it.
Private Information Retrieval (PIR)	Retrieving an entry from a database without revealing which entry was retrieved.
Relinearization keys	Evaluation keys that shrink a ciphertext back to normal size after a multiplication.
Trusted Authority (TA)	The party that holds the secret key, locates table indices and builds PIR queries. The original report calls it the utility provider or water center.

3. User Requirements

The requirements below are those of the original report. The last column records whether the implementation meets each one, based on a review of the code in October 2026 (Appendix E).

ID	Requirement	Met by	Status
UR-1	Encrypt usage data so that algorithms operate directly on ciphertexts and never need to decrypt an individual reading.	Meters encrypt in Step 1; every CS step works only on ciphertexts.	Met in design Some CS programs load the secret key for debugging.
UR-2	Encrypt consumption data at the point of collection, and compare the encrypted computation with the original data to measure accuracy.	Step 1 saves the plaintext ratio; `CheckRes` saves the decrypted encrypted ratio.	Met The two results are compared by hand.
UR-3	Use a homomorphic LUT approach, storing input/output pairs, to compute the AM/HM ratio on encrypted data.	`MakeEncTab_*` and Steps 1–7.	Met Table-size constants do not match the generated tables.
UR-4	Compute ratios with LUTs because BFV cannot divide, and simulate the communicating components: smart meters, data collectors, computation servers, LUT provider and utility provider.	Section 5.2.	Met Meters and the data collector are simulated inside Step 1.
UR-5	Demonstrate that encrypted computation preserves accuracy while maintaining privacy.	Accuracy comparison (Chapter 8).	Reported passed A full run of the committed code stops at Step 7.

4. System Requirements

Requirement	Version	Purpose and notes
Microsoft SEAL	4.1.2 exactly, shared library	Homomorphic encryption library; provides the BFV scheme.
CMake	3.10 or later	Build configuration.
OpenMP	18.1.8 (used in development)	Shared-memory parallelism across CPU cores.
C++ compiler	C++17: GCC 9.3+ or Clang	Builds every program.
Memory	8 GB or more	Encrypted tables and rotations are memory-intensive.
Operating system	macOS or Linux; WSL2 on Windows	Some files are opened in text mode, which breaks on native Windows.
Optional	GoogleTest 1.12.1, GoogleBenchmark 1.7.1	Testing SEAL itself.

The test plan adds non-functional goals: no plaintext or key leakage in logs or errors, defined performance thresholds, graceful handling of invalid input, and scalability to larger data sets. No numeric targets were set for these. Their status is discussed in Chapter 9.

5. System Architecture

5.1 System context

The system has five kinds of party. Smart meters in a region encrypt their readings and send them, through a data collector, to a single Computation Server. The LUT provider supplies encrypted lookup tables ahead of time. The Trusted Authority, the only holder of the secret key, helps the server with each table lookup and decrypts the final result. Every party communicates only with the Computation Server.

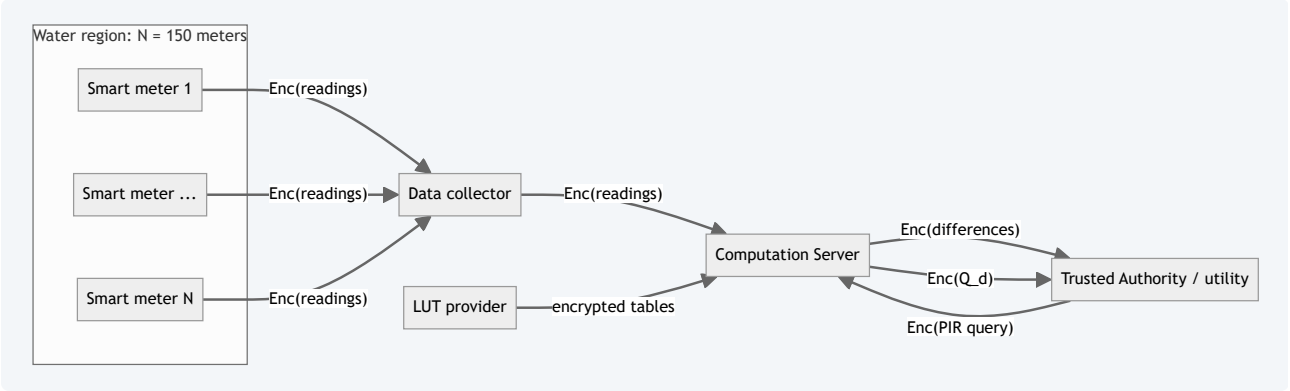


Figure 1. System context. All data crossing an arrow is encrypted.

5.2 Components

Component	In the code	Holds	Responsibilities
Smart meter	Simulated in `Step1_CS1`	Public key	Collects hourly consumption at the customer's premises and encrypts it before transmission.
Data collector	Simulated in `Step1_CS1`	Nothing secret	Gathers the ciphertexts from every meter in a region and forwards them to the CS.
Computation Server	`Step*_CS*`	Public, relinearization and Galois keys	Performs every homomorphic computation: sums, table searches, PIR retrievals and the final product.
LUT provider	`MakeEncTab_1/2/3`	Public key	Builds encrypted input and output tables for the operations BFV cannot compute: reciprocal, division and rescaling.
Trusted Authority / utility	`Step*_TA*`, `CheckRes`	Secret key	Decrypts table-search differences to locate an index, returns an encrypted PIR query, and decrypts and analyses the final ratio.

5.3 Keys and trust boundaries

`KeyGen` generates four keys from one set of parameters. Each party receives only the keys it needs:

Key	Purpose	Given to

Public key	Encryption	Smart meters, LUT provider, TA (to encrypt PIR queries)
Secret key	Decryption	TA only
Relinearization key	Shrinking ciphertexts after multiplication	CS
Galois keys	Rotating ciphertext slots for PIR and summation	CS

The secret key is the decisive trust boundary: whoever holds it can decrypt any ciphertext. The design keeps it away from the CS, so the server that processes every reading can read none of them. The TA holds the key but only ever receives aggregate differences, never an individual meter's ciphertext.

5.4 Protocol

The CS and TA take turns in seven numbered steps. Each lookup is a round trip: the CS sends encrypted differences, and the TA answers with an encrypted PIR query.

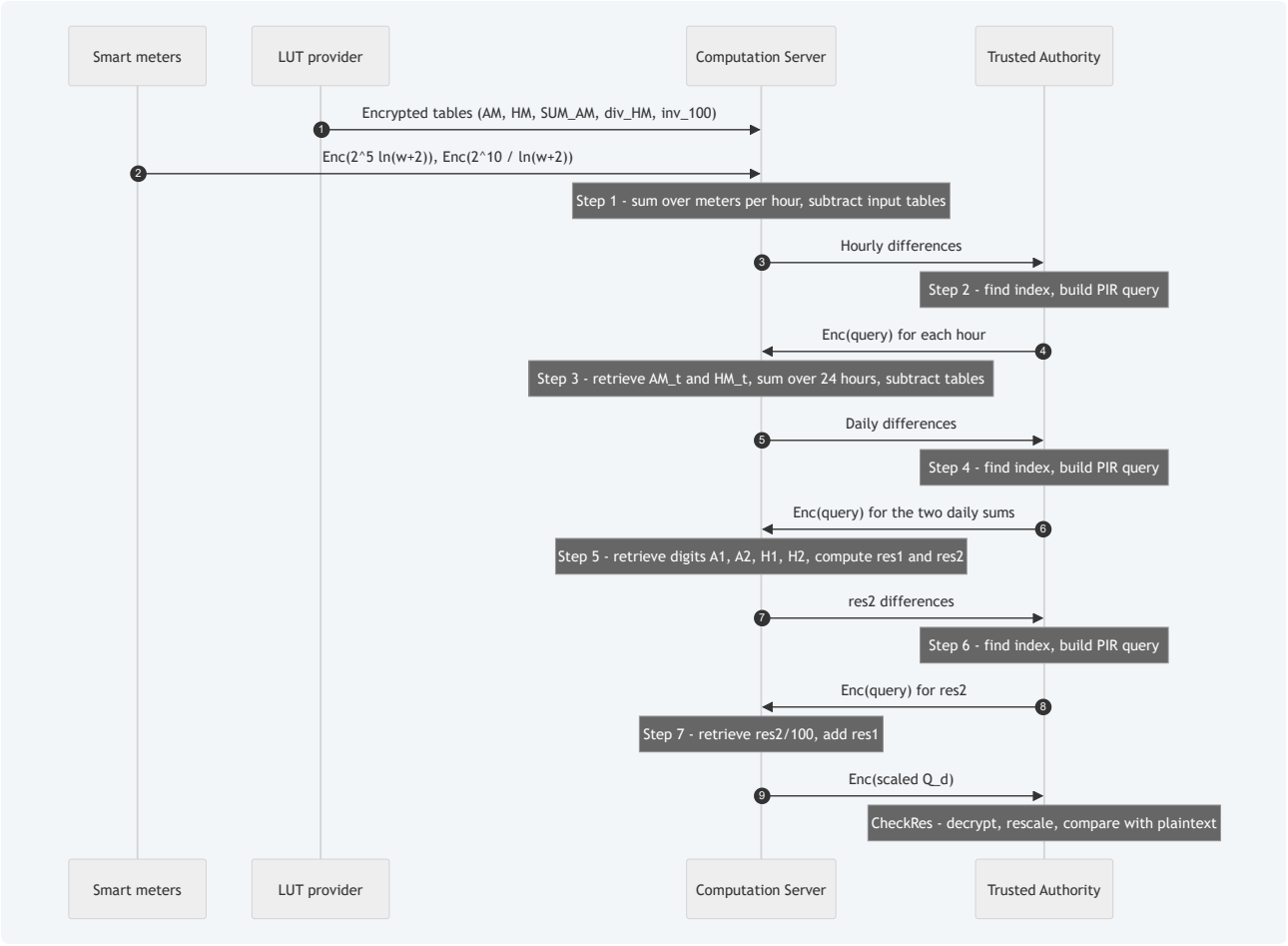


Figure 2. Message sequence for one day.

5.5 Lookup tables and Private Information Retrieval

BFV supports only addition, subtraction and multiplication, so division, reciprocals and rescaling use the LUT + PIR method:

1. **Table search.** The LUT provider publishes an encrypted input table of every value x_k a quantity can take, and an output table of $f(x_k)$. The CS subtracts the input table from the encrypted value v, slot by slot.
2. **Index lookup.** The TA decrypts the differences $v - x_k$. The slot that is zero, or where the sign changes, gives the index k^*.
3. **PIR query.** The TA encrypts a one-hot vector for the column of k^* and a rotated copy that encodes its row.
4. **Retrieval.** For each table row j, the CS rotates the row query by $-j$, multiplies it by the column query (only the matching row survives) and by output row j. Summing over rows and a logarithmic rotate-and-add place $\mathrm{Enc}(f(x_{k^*}))$ in every slot. The CS never learns k^*.

Each table holds 4096 values per ciphertext row (the first batch row of a 8192-degree plaintext). Longer tables span several rows, and the last row is padded with a sentinel that never matches.

5.6 Deployment of the simulation

All parties are simulated on one machine. Each step is a separate executable, and the parties exchange serialized SEAL objects through folders: `Key/` (parameters and keys), `Table/` (encrypted tables), `Result/` (differences, queries and intermediate ciphertexts), `ptxt_res/` (plaintext reference ratio) and `ctxt_res/` (decrypted encrypted ratio). Steps 1–7 use OpenMP for parallel loops. Two variants exist: `src/24hour`, the reference pipeline that processes a full day per step, and `src/1hour`, which processes one hour at a time and is incomplete. A production deployment would place each party on its own host, exchange ciphertexts over the network, and give each party only its own keys.

6. System Models

6.1 Notation

Symbol	Meaning	Value in the code
N	Number of smart meters in the region	150
T, t	Set of hourly time slots in a day; one slot	$\lvert T\rvert = 24$
d	One day	2014 data set
$w_i(t)$	Water consumption reported by meter i in slot t	input file
$x_i(t)$	Transformed reading, $\ln(w_i(t) + 2)$	`Step1_CS1`
AM_t, HM_t	Arithmetic and Harmonic Mean of slot t	hourly LUT outputs
Q_d	HM/AM ratio of day d	final result
Δ	Base of the digit split	100
t_p	BFV plain modulus	786433

6.2 Statistical model

For each time slot the Arithmetic and Harmonic Means are taken over the N meters:

$$AM_t = \frac{1}{N}\sum_{i=1}^{N} x_i(t) \qquad\qquad HM_t = \frac{N}{\sum_{i=1}^{N}\frac{1}{x_i(t)}}$$

The daily ratio sums the hourly means over the day. Because BFV cannot divide, it is evaluated as a product with a reciprocal obtained from a lookup table:

$$Q_d = \frac{\sum_{t=1}^{24} HM_t}{\sum_{t=1}^{24} AM_t} = \sum_{t=1}^{24} HM_t \times \frac{1}{\sum_{t=1}^{24} AM_t}$$

Why the ratio reveals tampering. For positive values $HM_t \le AM_t$, with equality only when every meter reports the same value. Therefore $0 < Q_d \le 1$, and Q_d measures how dispersed the readings are rather than how large. Scaling every averaged value by a common factor leaves Q_d unchanged. Additive and deductive attacks change the dispersion of the affected meters and move the ratio. A camouflage attack can keep AM_t fixed, but HM_t is more sensitive to small values, so the ratio still moves. The transform $x = \ln(w + 2)$ compresses large readings, and the offset of 2 keeps $x > 0$ so that $1/x$ is defined even for a zero reading. The anomaly detector that decides whether a given Q_d is abnormal is outside this project's scope.

6.3 Lookup-table model

Each non-polynomial function is a pair of tables (T_{in}, T_{out}). The original report's functions map to the generated tables as follows (2^9 is the output scale of the hourly tables):

Function	Quantity	Input x	Output $f(x)$	Tables
f_1	HM_t	$\sum_i 2^{10}/x_i(t)$	$2^9 \cdot N/(x/2^{10})$	`HM_input/output`
f_2	AM_t	$\sum_i 2^5\, x_i(t)$	$2^9 \cdot (x/2^5)/N$	`AM_input/output`
f_3, f_4	$\mathcal{H}_1, \mathcal{H}_2$	$\sum_t HM_t$ (scaled)	$\lfloor x/\Delta \rfloor$, $x \bmod \Delta$	`div_HM_*`
f_5, f_6	$\mathcal{A}_1, \mathcal{A}_2$	$\sum_t AM_t$ (scaled)	$v = 2^{21}/(x/2^9)$; $\lfloor v/\Delta \rfloor$, $v \bmod \Delta$	`SUM_AM_input`, `inv_SUM_AM_*`
f_7	res_2/Δ	res_2	$\mathrm{round}(x/\Delta)$	`inv_100_*`

6.4 Polynomial representation

The scaled product $\sum HM_t \times 1/\sum AM_t$ does not fit in the plaintext space, so both factors are written as two base-Δ digits:

$$\sum_{t=1}^{24} HM_t = \mathcal{H}_1\Delta + \mathcal{H}_2 \qquad \frac{1}{\sum_{t=1}^{24} AM_t} = \mathcal{A}_1\Delta + \mathcal{A}_2$$

$$(\mathcal{H}_1\Delta + \mathcal{H}_2)(\mathcal{A}_1\Delta + \mathcal{A}_2) = \mathcal{H}_1\mathcal{A}_1\Delta^2 + (\mathcal{H}_1\mathcal{A}_2 + \mathcal{H}_2\mathcal{A}_1)\Delta + \mathcal{H}_2\mathcal{A}_2$$

Dividing by $\Delta^2 = 10\,000$ and discarding the last, least significant term gives

$$res_1 = \mathcal{H}_1\mathcal{A}_1, \qquad res_2 = \mathcal{H}_1\mathcal{A}_2 + \mathcal{H}_2\mathcal{A}_1, \qquad \widetilde{Q}_d = res_1 + \frac{res_2}{\Delta}$$

Step 5 computes res_1 and res_2. Steps 6 and 7 evaluate res_2/Δ with one more lookup and add res_1. $\Delta = 100$ is a design choice: a larger plaintext space keeps more digits but costs execution time.

6.5 Encoding and precision

BFV operates on integers modulo t_p, so every real quantity is scaled by a power of two and rounded. Integer encoding is used instead of bitwise encoding because it is much faster.

Stage	Encoded value
Per-meter AM input	$\mathrm{round}(2^5\, x_i(t))$
Per-meter HM input	$\mathrm{round}(2^{10}/x_i(t))$
Hourly AM_t, HM_t	scaled by 2^9
Reciprocal of the daily AM sum	$2^{21}/\sum_t AM_t$
Product	$2^9 \sum_t HM_t \cdot 2^{21}/\sum_t AM_t = 2^{30} Q_d$
Decrypted result	$\widetilde{Q}_d \approx 2^{30} Q_d/10^4$, so $Q_d = \widetilde{Q}_d \cdot 10^4/2^{30}$

The tables cover readings from about 50 to 6000, so $x \in [\ln 52, \ln 6002] \approx [3.95, 8.70]$. The resulting ranges show why the digit split is necessary and that it suffices:

Quantity	Approximate range	Fits in $t_p = 786433$?
$\sum_t AM_t$ (real value)	95 – 209	yes
$2^{21} / \sum_t AM_t$, giving $\mathcal{A}_1$	10 000 – 22 000; $\mathcal{A}_1 \approx 100 – 220$	yes
$2^9 \sum_t HM_t$, giving $\mathcal{H}_1$	48 600 – 107 000; $\mathcal{H}_1 \approx 486 – 1070$	yes
Undivided product $2^{30} Q_d$	up to $\approx 1.07 \times 10^9$	**no**, hence the digit split
res_1	up to $\approx 235\,000$	yes
res_2	up to $\approx 128\,000$	yes
$\widetilde{Q}_d$	up to $\approx 107\,000$	yes

6.6 Error budget

The encrypted result differs from the plaintext ratio because of input rounding (to 2^{-5} and 2^{-10}), the integer rounding of each table output, and the digit split, which drops $\mathcal{H}_2\mathcal{A}_2/10^4 < 1$ and rounds res_2/Δ by at most 0.5. One unit of $\widetilde{Q}_d$ equals $10^4/2^{30} \approx 9.3 \times 10^{-6}$ in Q_d, so the digit split contributes an error of order 10^{-5}. Input rounding and the hourly output scale dominate the rest. Three precision levels exist (`MakeEncTab_1/2/3`, with hourly scales 2^9, 2^7 and 2^5): lower precision gives smaller tables and faster runs at the cost of accuracy.

6.7 FHE parameters and noise

Parameter	Value	Effect
Scheme	BFV	Exact integer arithmetic modulo t_p
Polynomial modulus degree	8192	8192 slots in two rows of 4096; 128-bit security
Coefficient modulus	BFV default for 8192 (≈ 218 bits)	Noise budget for multiplications
Plain modulus	786433 (prime, ≡ 1 mod 16384)	Enables batching; bounds every encoded value

Each ciphertext multiplication consumes much of the noise budget. A LUT retrieval multiplies the rotated row query by the column query and by an output-table row. Its result depends only on the fresh query and table ciphertexts, not on the value being looked up. **Every lookup therefore resets the multiplicative depth**: the noise accumulated in $\mathrm{Enc}(v)$ is used only for the differences the TA decrypts. The deepest chain is two multiplications for a retrieval plus one for res_1 and res_2, within the default parameters. BFV relinearization keeps ciphertexts small after each multiplication, and Galois keys provide the rotations.

6.8 Threat model

Party	Assumed behaviour	What it learns

Smart meter	Honest; public key only	Its own readings
Data collector	Honest-but-curious	Ciphertexts only
Computation Server	Honest-but-curious; evaluation keys	Ciphertexts only; PIR hides which table entry was used
LUT provider	Honest; public key	The table ranges it builds
Trusted Authority	Honest-but-curious; sole secret-key holder	The decrypted differences. Because tables are sorted, it learns the **aggregate** values (hourly sums over N meters, the daily sums, res_2 and Q_d), but never an individual reading

The system thus protects individual readings from every party, and all values from the Computation Server. It assumes the CS and TA do not collude. Hiding the aggregates from the TA as well would require randomly rotating the tables before each search, a possible extension (Section 9.3).

7. System Evolution

When	Milestone
Before May 2024	Concept: privacy-preserving analytics for smart water; FHE chosen at the client's direction
May 2024	Repository created from the smart-energy codebase
July 2024	Implementation finished: 24-hour and 1-hour pipelines
August 2024	Generated keys, tables and binaries removed from version control
Nov – Dec 2024	Step 7 and Step 1 fixes; project report, test report and test plan
July 2025	Updated report published
October 2026	Code review, repository documentation and this revised report

Table 7.1. Project timeline, from the repository history.

7.1 Initial concept

The project began with the goal of protecting customer water consumption data in a smart water grid while still allowing the utility to analyse it. Several privacy techniques were available. Secure Multiparty Computation requires many rounds of interaction and scales poorly to many meters. Differential Privacy adds noise that degrades the accuracy of the detection ratio. Fully Homomorphic Encryption gives an exact result on encrypted data and was the approach the client requested.

7.2 Inheriting the smart-energy implementation

The team was given an existing codebase for smart *electricity* grids that encrypted per-meter usage, computed the HM/AM ratio with LUT + PIR, and decrypted the result. The water system kept its architecture: the CS and TA alternating in numbered steps, BFV in Microsoft SEAL, and lookup tables for division. Traces of this origin remain, such as the header name `SGSimulation.hpp` ("smart grid simulation").

7.3 Dataset shift

Moving from energy to water required reading a new data format (one file per day, one row per hour, one column per meter, $N = 150$), regenerating the lookup tables for the range of water readings, and keeping the table-size constants consistent with the new tables. The data set covers 2014 and is not redistributed: the team had permission to use it only for a specific period.

7.4 Two pipeline variants

The 24-hour variant processes a whole day in each step and is driven by a script that iterates over the year. The 1-hour variant runs the first steps once per hour with a running sum, to model meters that re-

port hourly. It splits Steps 3 and 5 into separate AM and HM programs. The 24-hour variant became the reference; the 1-hour variant was not completed.

7.5 Precision experiments

Three table generators implement the same tables at three precision levels, to study the trade-off between accuracy and execution time described in Section 6.6.

7.6 Testing and validation

Accuracy was validated by comparing the decrypted encrypted ratio with the ratio computed on plaintext data, supported by unit and metamorphic tests (Chapter 8).

7.7 Design decisions

Decision	Alternatives	Reason
BFV scheme	CKKS; bitwise schemes such as TFHE	Exact integer results with batching. CKKS is approximate; bitwise schemes are far slower.
Integer encoding with power-of-two scaling	Bitwise encoding	Much faster; precision is tuned through the scaling exponents.
LUT + PIR for division and reciprocal	Polynomial approximation of $1/x$	Exact table values, and each lookup resets the multiplicative depth.
Base-100 digit split	Larger plain modulus	The undivided product exceeds t_p; a larger modulus costs noise budget and time.
$\ln(w+2)$ transform	Raw readings	Compresses the range of readings, keeping tables small, and avoids division by zero.
One executable per step, file exchange	Single program; networked service	Mirrors the multi-party model; each step can be timed and rerun.
Polynomial degree 8192	4096; 16384	Enough noise budget for three multiplications at 128-bit security, with 4096 slots per table row.

7.8 Summary of evolution

The project moved from a proof of concept, through adoption of the smart-energy implementation, to a water-specific pipeline with its own data handling, tables and validation. Its central technical challenge, computing a ratio under a scheme that cannot divide while staying inside a small plaintext space, was met by combining lookup tables, PIR and the digit-split representation.

8. Testing and Validation

8.1 Accuracy validation

Every run produces two values of Q_d for the same day. Step 1 computes a plaintext reference in double precision, and `CheckRes` decrypts and rescales the result of the encrypted pipeline. Their difference measures the error introduced by encoding and the lookup tables (Section 6.6).

8.2 Unit tests

#	Under test	What was verified
1	`OutputPlaintext`	Decrypted Step 2 results are written in the expected format.
2	`OutVector`	Encoded Step 1 vectors are printed in full; output depends on the configured table sizes.
3	`PrintParameters`	Each step's SEAL context carries the expected BFV parameters.
4	`ShiftWork` (index form)	The PIR row query is shifted correctly. The random form was not tested.
5	`LoadKey`	Keys load into a context created from the saved parameters.
6	`KeyGen`	All four keys are generated, saved and loadable from any context with the same parameters.

8.3 Metamorphic tests

Relation	Expectation	Result
Computational accuracy	AM, HM and Q_d computed on ciphertexts equal the plaintext values.	Passed
Scaling	Scaling the inputs scales AM and HM alike and leaves Q_d unchanged.	Passed

> The scaling relation holds exactly for the averaged values x. Because the code averages $x = \ln(w + 2)$, scaling the raw readings w shifts x instead and does change Q_d. A repeat of this test should scale x.

8.4 Planned tests

The test plan also describes unit tests (encrypt and decrypt 42; add 42 + 58; multiply 6 × 7; reject a corrupted ciphertext), integration tests (one meter over a week, many meters, mixed operations), and performance and regression tests. These were described as scripts under `build/tests/`, which are not part of the repository; there is no continuous integration.

8.5 Recommended additional tests

- An automated end-to-end run on one day, comparing plaintext and encrypted Q_d within a tolerance.
- A build-time check that the table-size constants match the generated tables.
- Index-search tests where the value is missing, falls in the last slot, or matches twice.
- Error and run-time measurements for each precision level and several meter counts.
- Synthetic additive, deductive and camouflage attacks, to show that Q_d moves under attack.

9. Conclusion

This project demonstrates a privacy-preserving approach to analysing smart water consumption with Fully Homomorphic Encryption. Using the BFV scheme with lookup tables and Private Information Retrieval, it computes the Arithmetic Mean, the Harmonic Mean and their daily ratio directly on encrypted data, so that each customer's readings stay encrypted from the meter to the final result.

9.1 Key achievements

- **Privacy preservation.** Meters encrypt at the source. The Computation Server never holds the secret key, and the Trusted Authority sees only aggregates, never an individual reading.
- **Operations beyond BFV.** Division, reciprocals and rescaling are evaluated with encrypted lookup tables, and PIR hides from the server which entry was used.
- **Accuracy within a small plaintext space.** The digit-split representation keeps every intermediate value below the plain modulus at an error of order 10^{-5} in the ratio.
- **Bounded noise.** Each lookup restarts from fresh ciphertexts, so depth stays at three multiplications and standard 128-bit parameters suffice.
- **Built-in validation.** Every run yields both plaintext and encrypted results, and the reported tests found them consistent.

9.2 Limitations

- The Trusted Authority learns aggregate sums, because tables are searched in sorted order.
- The simulation runs every party on one machine with a shared key folder; party separation is by convention.
- Correctness issues found in the October 2026 review prevent a complete run of the committed code, and the 1-hour variant is incomplete (Appendix E).
- The system computes the detection ratio but does not include a detector that classifies it.
- Tables are built for 150 meters and a fixed reading range, and results come from one data set for one year.
- Accuracy and run time were not quantified, and tests are not automated.

9.3 Future direction

The original report identified three directions: scaling the LUT and PIR integration to larger data sets, computing more advanced metrics, and extending the approach to other industries. A practical path towards them:

1. Resolve the known correctness issues and add an automated end-to-end accuracy test.
2. Quantify error and run time across precision levels and meter counts.
3. Separate the parties: their own keys, no secret key on the server, network transport.

4. Randomly rotate tables before each search, so that the TA no longer learns aggregates.
5. Add an anomaly detector on Q_d and evaluate it against synthetic attacks.
6. Consolidate the duplicated 1-hour and 24-hour code.
7. Compare the BFV pipeline with an approximate scheme such as CKKS.

The project's central idea is sound: lookup tables and PIR give a BFV pipeline the division it lacks while keeping noise bounded. With the steps above, the prototype can become a basis for measuring how far privacy-preserving analytics on smart water grids can scale.

Appendix A: Key Generation

Key	Purpose	Usage in this project
Public key	Encrypts data at the point of collection, so no one can see the unencrypted data.	Shared with smart meters across the network, the LUT provider and the TA.
Secret key	Decrypts data after computations on ciphertexts.	Retained securely by the authorised decrypting party, the TA / utility provider.
Relinearization key	Reduces ciphertext size after multiplication, which otherwise raises the ciphertext's degree.	Used by the CS after each multiplication.
Galois keys	Enable SIMD operations such as slot rotations on encrypted vectors.	Used by the CS for PIR retrieval and rotate-and-add sums.

The keys are saved in the `Key/` directory with the encryption parameters and can be loaded from any context that uses the same parameters, so they are not regenerated for every step. Regenerating keys makes every existing encrypted table unusable, so tables must be rebuilt afterwards.

Appendix B: System Setup

Required software is listed in Chapter 4; SEAL 4.1.2 must match exactly. To build the reference pipeline:

```
cd src/24hour
rm -f CMakeCache.txt Makefile cmake_install.cmake
cmake . && make -j
mkdir -p Key Table Result ptxt_res ctxt_res
./bin/KeyGen
./bin/MakeEncTab_1
# then Steps 1-7 and CheckRes for each day (script.py)
```

If CMake cannot find the installed software, open `CMakeLists.txt` in the variant folder and, under the comment `# Import OpenMP`, set the install paths, correcting every path. The repository's `docs/BUILD.md` and `docs/USAGE.md` give platform-specific instructions and the input data format.

Appendix C: Technical Implementation

File	Party	Purpose
`KeyGen.cpp`	TA	Sets BFV parameters; generates and saves all four keys.
`MakeEncTab_1/2/3.cpp`	LUT	Generates the encrypted input and output tables at three precision levels.
`Step1_CS1.cpp`	CS	Encodes and encrypts readings (simulating the meters), sums them per hour, subtracts the AM/HM input tables, and saves the plaintext reference ratio.
`Step2_TA1.cpp`	TA	Decrypts hourly differences, finds the table indices, encrypts the PIR queries.
`Step3_CS2.cpp` (1-hour: `_1` AM, `_2` HM)	CS	Retrieves AM_t and HM_t with PIR, sums over the day, subtracts the daily input tables.
`Step4_TA2.cpp`	TA	Locates the daily sums in their tables and encrypts the PIR queries.
`Step5_CS3.cpp` (1-hour: `_1`, `_2`)	CS	Retrieves the digits $\mathcal{A}_1, \mathcal{A}_2, \mathcal{H}_1, \mathcal{H}_2$, computes res_1 and res_2, subtracts the `inv_100` input table.
`Step6_TA3.cpp`	TA	Locates res_2 in its table and encrypts the PIR query.
`Step7_CS4.cpp`	CS	Retrieves $res_2/100$, adds res_1, saves the encrypted scaled ratio.
`CheckRes.cpp`	TA	Decrypts and rescales the final ratio for comparison with the plaintext reference.
`SGSimulation.hpp`, `Utility.hpp`	—	Constants (meter count, table sizes, precision, file paths) and shared helpers (key loading, data reading, PIR shifts).

Appendix D: Original Report and Implementation

Topic	Original report	Implementation (described in this edition)
Value averaged	Raw reading $w_i(t)$	$\ln(w_i(t) + 2)$
Precision parameter	A single 2^p	2^5 (AM inputs), 2^{10} (HM inputs), 2^9 (hourly outputs), 2^{21} (reciprocal)
f_5	$x/100$	Reciprocal of the daily AM sum, then split into digits
Final step	`Step7_TA4`	`Step7_CS4`, run by the Computation Server
Decrypting party	Utility provider / water center	Trusted Authority (same role)
Architecture figure	Four boxes, one-way arrows	Figure 1 adds the CS–TA lookup exchange

Appendix E: Known Issues (October 2026 Review)

A review of the repository found the following, none of which are fixed yet. The full list with file locations is in `docs/KNOWN_ISSUES.md`.

Area	Issue
Table sizes	The `TABLE_SIZE_*` constants are smaller than the generated tables, so readers load only the first row of multi-row tables.
Step 7 (24-hour)	Reads `Fin_AM1HM1` without the underscore that Step 5 writes, so the final step cannot find its input.
Index search	Only the inner loop is exited on a match, row-end comparisons read the wrong batch row, and an unmatched value still produces a query.
1-hour pipeline	Several wrong file names and table names and an uninitialised loop counter; the pipeline does not complete.
Hour ordering	Hours are keyed by text, so "10" sorts before "2".
Driver script	Leftover `exit(0)` calls stop the run after the first step of the first day.
Security	Some CS programs load the secret key for debugging; all parties share one key folder; old keys exist in the git history.
Engineering	Hard-coded paths in CMake, generated files under version control, duplicated code, no automated tests.

Bibliography

1. Smart Grid Detection Simulation.
2. IEEE Privacy Preserving Analytics Publication.
3. S. Bhattacharjee, privacy-preserving analytics paper.
4. S. Bhattacharjee, forensics paper.
5. Detection of False Data Injection in Smart Water Metering Infrastructure.
6. Microsoft SEAL (release 4.1), Microsoft Research.
7. J. Fan and F. Vercauteren, "Somewhat Practical Fully Homomorphic Encryption," IACR Cryptology ePrint Archive, 2012/144.

Entries 1–5 are as listed in the original report, which does not give full citations.